arXiv:2310.15213v2 [cs.CL] 25 Feb 2024

FUNCTION VECTORS IN LARGE LANGUAGE MODELS

Eric Todd[*], **Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau**
Khoury College of Computer Sciences, Northeastern University

ABSTRACT

We report the presence of a simple neural mechanism that represents an input-output function as a vector within autoregressive transformer language models (LMs). Using causal mediation analysis on a diverse range of in-context-learning (ICL) tasks, we find that a small number attention heads transport a compact representation of the demonstrated task, which we call a *function vector* (FV). FVs are robust to changes in context, i.e., they trigger execution of the task on inputs such as zero-shot and natural text settings that do not resemble the ICL contexts from which they are collected. We test FVs across a range of tasks, models, and layers and find strong causal effects across settings in middle layers. We investigate the internal structure of FVs and find while that they often contain information that encodes the output space of the function, this information alone is not sufficient to reconstruct an FV. Finally, we test semantic vector composition in FVs, and find that to some extent they can be summed to create vectors that trigger new complex tasks. Our findings show that compact, causal internal vector representations of function abstractions can be explicitly extracted from LLMs.

1 INTRODUCTION

Since the study of the lambda calculus (Church, 1936), computer scientists have understood that the ability for a program to carry references to its own functions is a powerful idiom. Function references can be helpful in many settings, allowing expression of complex control flow through deferred invocations (Sussman, 1975), and enabling flexible mappings from inputs to a target task. In this paper we report evidence that autoregressive transformers trained on large corpora of natural text develop a rudimentary form of function references.

Our results begin with an examination of in-context learning (ICL; Brown et al., 2020). ICL mechanisms have previously been studied from the perspective of making copies (Olsson et al., 2022) and from a theoretical viewpoint (Von Oswald et al., 2023; Garg et al., 2022; Dai et al., 2023), but the computations done by large models to generalize and execute complex ICL functions are not yet fully understood. We characterize a key mechanism of ICL execution: *function vectors* (FVs), which are compact vector representations of input-output tasks that can be found within the transformer hidden states during ICL. An FV does not directly perform a task, but rather it triggers the execution of a specific procedure by the language model (Figure 1).

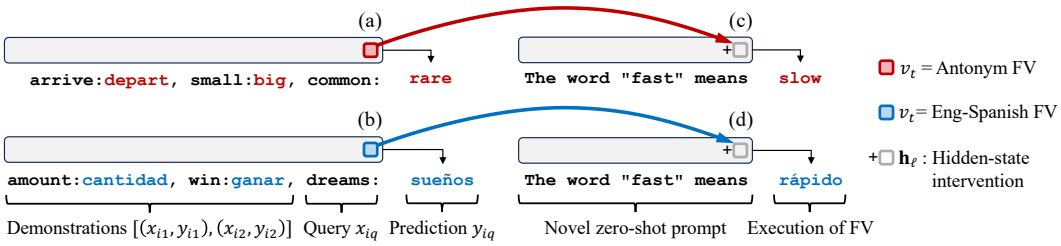

Figure 1: An overview of **function vectors** (FVs). An FV is extracted from activations induced by in-context examples of (a) antonym generation or (b) English to Spanish translation, and then inserted into an unrelated context to induce generation of (c) a new antonym or (d) translation.

[*]Correspondence to todd.er@northeastern.edu. Open-source code and data available at functions.baulab.info.

Function vectors arise naturally when applying causal mediation analysis (Pearl, 2001; Vig et al., 2020; Meng et al., 2022; 2023; Wang et al., 2022a) to identify the flow of information during ICL. We describe an activation patching procedure to determine the presence of a handful of attention heads that mediate many ICL tasks. These heads work together to transport a function vector that describes the task; the FV can be formed by summing outputs of the causal attention heads.

We test the hypothesis that function vectors are a general mechanism spanning many types of functions. To quantify the role and efficacy of function vectors, we curate a data set of over 40 diverse ICL tasks of varying complexity. We calculate FVs for these tasks and investigate impact of FVs in triggering those functions across a variety of LMs scaling up from 6B to 70B parameters.

We further ask whether FVs are portable: are the effects of an FV limited to contexts very similar to those where it is extracted, or can an FV apply in diverse settings? We compare the effects of FVs when inserted into diverse input contexts including differently-formatted forms, zero-shot formats, and natural text contexts. We find that FVs are remarkably robust, typically triggering function execution even in contexts that bear no resemblance to the original ICL context.

A key question is whether the action of FVs can be explained by word-embedding vector arithmetic (Mikolov et al., 2013; Levy & Goldberg, 2014; Merullo et al., 2023). We examine decodings of FVs (Nostalgebraist, 2020), and find that although FVs often encode a function's output vocabulary, those vocabularies do not fully identify an FV. In other words, to invoke functions, FVs need to carry some additional information beyond their encoding of the top vocabulary words.

Finally, we investigate whether the space of FVs has its own vector algebra over functions rather than words. We construct a set of composable ICL tasks, and we test the ability of FVs to obey vector algebra compositions. Our findings reveal that, to some extent, vector compositions of FVs produce new FVs that can execute complex tasks that combine constituent tasks. We emphasize that FV vector algebra is distinct from semantic vector algebra over word embeddings: for example, composed FV vectors can specify nonlinear tasks such as calculating the antonym of a word, that cannot themselves be implemented as a simple embedding-vector offset (Appendices A, L).

2 Method

2.1 A Motivating Observation

When a transformer processes an ICL prompt with exemplars demonstrating task t, do any hidden states encode the task itself? We seek causal features rather than just correlations.

We can investigate this question with the following simple test: Gather a set of ICL prompts P_t for the task t and compute the average activation $\bar{\mathbf{h}}_\ell^t$ at the last token of each prompt at a particular layer ℓ of the model (Figure 2a). Then perform an intervention where $\bar{\mathbf{h}}_\ell^t$ is added to the representation after ℓ when the transformer completes a previously unseen zero-shot prompt (Figure 2b).

Surprisingly, we find that adding the average activations in this way at particular layers induces the model to perform the task in the new context. For example, if $t =$ *antonym*, the red line in Figure 2c shows that adding $\bar{\mathbf{h}}_{12}^t$ at layer 12 in GPT-J causes the model to produce antonyms in a zero-shot context, with 24.3% accuracy. That suggests that $\bar{\mathbf{h}}_{12}^t$ does encode the antonym task.

The effect of $\bar{\mathbf{h}}_\ell^t$ leads us to ask: Can we distill a more effective hidden-state representation of the task t? In the rest of Section 2 we describe an analysis of the mechanisms of ICL that leads to a function vector representation v_t whose stronger causal effects are shown as a green line in Figure 2c.

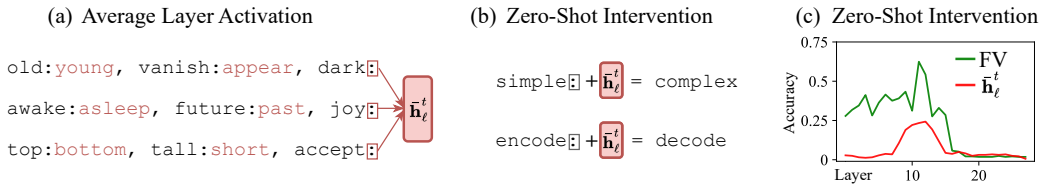

Figure 2: A motivating observation: (a) an average activation is computed over a set of antonym ICL prompts, and (b) added to a zero-shot context, which produces the opposite of unseen words. (c) Systematic effects (in red) for adding $\bar{\mathbf{h}}_\ell^t$ in middle layers of the network; even stronger effects are seen by the FV (in green).

2.2 FORMULATION

An autoregressive transformer language model f takes an input prompt p and outputs a next-token distribution $f(p)$ over vocabulary $\mathcal{V}$; we write $f(p)[y] \in [0, 1]$ for the predicted probability of output $y \in \mathcal{V}$ in response to input p. Internally, f comprises L layers; we examine their calculations at the last token position. Each layer $\ell \leq L$ has a vector representation of the last token, $\mathbf{h}_\ell \in \mathbb{R}^d$, that is computed from the previous layer as $\mathbf{h}_\ell = \mathbf{h}_{\ell-1} + m_\ell + \sum_{j \leq J} a_{\ell j}$, where m_ℓ is the output of a multilayer perceptron, and $a_{\ell j}$ is the projection of the output of the jth attention head (out of J heads) into the hidden state at layer ℓ. This definition of $a_{\ell j} \in \mathbb{R}^d$ adopts the framing of Elhage et al. (2021) rather than that of Vaswani et al. (2017) (see Appendix B for details). Attention heads and hidden states can be viewed as functions of transformer input, so we shall write $a_{\ell j}(p)$ or $\mathbf{h}_\ell(p)$ to denote their values when the transformer processes input p. The transformer's decoder D maps the last layer hidden state to the output distribution $D(\mathbf{h}_L(p)) = f(p)$.

For each task $t \in \mathcal{T}$ in our universe of ICL tasks $\mathcal{T}$ we have a data set P_t of in-context prompts $p_i^t \in P_t$. Each prompt p_i^t is a sequence of tokens with N input-output exemplar pairs (x, y) that demonstrate the same underlying task t mapping between x and y, and one *query* input x_{iq} corresponding to a target (correct) response y_{iq} that is not part of the prompt, that should be predicted by the LM if it generalizes correctly. We focus our analysis on successful ICL by including in P_t only prompts p_i^t where the prediction $f(p_i^t)$ ranks the correct answer y_{iq} highest. We write one ICL prompt as

$$p_i^t = [(x_{i1}, y_{i1}), \cdots, (x_{iN}, y_{iN}), x_{iq}] \tag{1}$$

We also make use of uninformative ICL prompts $\tilde{p}_i^t \in \tilde{P}_t$ for which the labels are shuffled; we use the tilde to indicate a shuffled prompt $\tilde{p}_i^t = [(x_{i1}, \tilde{y}_{i1}), \cdots, (x_{iN}, \tilde{y}_{iN}), x_{iq}]$ in which there is no systematic relationship between any of the x_{ik} and $\tilde{y}_{ik}$.

2.3 CAUSAL MEDIATION TO EXTRACT FUNCTION VECTORS FROM ATTENTION HEADS

To distill the information flow during ICL, we apply causal mediation analysis.

Given a transformer model f and an ICL prompt $p_i^t \in P_t$ from a dataset representing task t, we prompt the model with only input-output pairs (x_i, y_i). Therefore, the LM must infer the implicit relationship between these (x, y) pairs to correctly predict the answer given a novel query x_{iq}. We seek to identify model components with a causal role in the prediction of y_{iq}. We restrict our analysis to the attention heads since those are the components used by transformer LMs to move information between different token positions (Vaswani et al., 2017; Elhage et al., 2021). Formally, for each attention head $a_{\ell j}$ and task dataset P_t, we take the mean of task-conditioned activations $\bar{a}_{\ell j}^t$ as

$$\bar{a}_{\ell j}^t = \frac{1}{|P_t|} \sum_{p_i^t \in P_t} a_{\ell j}(p_i^t). \tag{2}$$

We then run the model on an uninformative ICL prompt $\tilde{p}_i^t \in \tilde{P}_t$ where each x is matched with a *random* output $\tilde{p}_i^t = [(x_i, \tilde{y}_i)]$. Now, the model is less likely to generate the correct output y_q as it cannot infer the relationship from incorrect ICL examples (notwithstanding the observation from Min et al. (2022) that some tasks can be guessed from incorrect labels). While running the model on $\tilde{p}_i^t$, we replace an attention head activation $a_{\ell j}$ with mean task-conditioned activation $\bar{a}_{\ell j}^t$ (Eq. 2) and measure its causal indirect effect (CIE) towards recovering the correct answer y_q as

$$\text{CIE}(a_{\ell j} \mid \tilde{p}_i^t) = f(\tilde{p}_i^t \mid a_{\ell j} := \bar{a}_{\ell j}^t)[y_{iq}] - f(\tilde{p}_i^t)[y_{iq}]. \tag{3}$$

The intuition here is to measure the degree to which using the "correct" mean attention head output $\bar{a}_{\ell j}^t$—computed over the *uncorrupted* prompts for task t—increases the mass assigned to the target response y_{iq}, relative to the likelihood of this token under the corrupted prompt $\tilde{p}_i^t$. A larger value implies that the corresponding head is more influential in promoting the correct response.

Then each attention head's average indirect effect (AIE) is calculated by averaging this difference across all tasks $t \in \mathcal{T}$ and (corrupted) prompts:

$$\text{AIE}(a_{\ell j}) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \frac{1}{|\tilde{P}_t|} \sum_{\tilde{p}_i^t \in \tilde{P}_t} \text{CIE}(a_{\ell j} \mid \tilde{p}_i^t) \tag{4}$$